EDGEVIA

Software Requirements Specification & System Architecture Blueprint

Version 1.0.0 — Initial Release	Date April 2026
Author Humayun Kabir	Status Active Development

1. Project Overview

EDGEVIA is a high-performance, multi-tenant reverse proxy and traffic management platform built for e-commerce businesses, SaaS startups, and any web service that experiences sudden traffic spikes — such as flash sales, product drops, ticket releases, and viral events.

FairFlow sits between the internet and the customer's origin server. When traffic exceeds the configured threshold, FairFlow activates a Virtual Waiting Room — holding users in a fair, branded queue instead of letting the server crash. The customer's business continues running. Revenue is protected.

1.1 The Problem

Small and mid-size businesses cannot afford Cloudflare Enterprise ($$$)

Shared hosting and budget VPS servers crash under sudden traffic

spikes. Server crashes during flash sales = direct revenue loss

Existing solutions are complex, expensive, or enterprise-only

1.2 The Solution

EDGEVIA provides enterprise-grade traffic protection at startup pricing. One CNAME change. Automatic SSL. Real-time dashboard. Fair queue for customers.

1.3 Target Market

Segment	Example	Pain Point
E-commerce (Shopify/ WooCommerce)	Fashion drops, sneaker releases	Server crash = lost sales
Ticketing Platforms	Concert/event ticket sales	Bot traffic, fair queue needed
SaaS on Product Hunt launch	New product launch day	10x traffic spike overnight
Custom VPS businesses	Any self-hosted service	No CDN protection layer

2. Business Model & Pricing

2.1 Pricing Strategy

FairFlow uses a hybrid model: flat monthly subscription + event-based add-ons.

Plan	Monthly Price	Included	Overage
Starter	$29/mo	3 sites, 1M req/mo, basic waiting room	$0.002/1K req
Growth	$79/mo	10 sites, 5M req/mo, custom branding, analytics	$0.0015/1K req
Business	$199/mo	Unlimited sites, 20M req/mo, priority support, API access	$0.001/1K req
Event Add on	$49/event	Dedicated event mode, real-time queue dashboard	Included

2.2 Revenue Streams

Monthly SaaS subscriptions (predictable MRR)

Event-based add-ons per flash sale/product drop

Overage billing per 1K requests beyond the plan limit

Enterprise custom contracts (future)

3. System Architecture

3.1 Architecture Overview

EDGEVIA uses a microservices architecture deployed on Kubernetes. Three primary services communicate via gRPC internally. External traffic flows exclusively through the Go Proxy Engine.

Pattern	**Polyglot microservices — Go for performance-critical proxy, Node.js for business logic, Next.js for UI**

Deployment	**Kubernetes (K8s) from Day 1 — Docker Compose for local development only**

Communication	**gRPC (internal service-to-service) + Socket.io (real-time dashboard push)**

3.2 Technology Stack

Layer	Technology	Justification
Proxy Engine	Go (Golang)	goroutines handle 50K+ concurrent connections, sub-ms latency
Management API	Node.js + NestJS	Scalable, typed, modular — handles auth/billing/ config
Frontend Dashboard	Next.js 14 (App Router)	SSR for first load, real-time via Socket.io hooks
Primary Database	PostgreSQL + Prisma ORM	ACID compliance for billing, sites, tenant data

Cache + Queue	Redis 7	Token bucket rate limiting, BullMQ job queue, pub/ sub
Job Queue	BullMQ (Node.js)	Async jobs: analytics aggregation, email, billing events
Internal Comms	gRPC + Protocol Buffers	Type-safe, binary protocol between Go and Node.js
Real-time UI	Socket.io	Live metrics push from API to dashboard
SSL Automation	Let's Encrypt + autocert (Go)	Auto-provision SSL certs per customer domain
Metrics	Prometheus + Grafana	Operational visibility, alerting
Logging	Zap (Go) + Pino (Node.js)	Structured JSON logging, low overhead
Tracing	OpenTelemetry	Distributed tracing across proxy + API
Container	Docker + Kubernetes	Production orchestration, auto-scaling
Object Storage	AWS S3 / Cloudflare R2	Waiting room assets, logos, analytics exports
Billing	Stripe	Subscriptions, usage metering, webhooks
Email	Resend	Transactional email (alerts, invoices, onboarding)
DNS	Cloudflare API	Programmatic DNS for customer CNAME setup

4. Feature Specifications

4.1 Core Proxy Engine (The Shield)

4.1.1 Zero-Configuration Onboarding

Customer adds a CNAME record pointing their domain to the FairFlow Go proxy,

and the new domain is automatically detected via SNI (Server Name Indication).

The Let's Encrypt SSL certificate is auto-provisioned within 60 seconds. Origin server URL stored in PostgreSQL, cached in Redis

4.1.2 Rate Limiting — Token Bucket Algorithm

Each tenant gets an isolated token bucket in Redis:

Key pattern: tenant:{tenantId}:bucket:{domain}

Bucket refills at configured RPS (requests per second) rate

When the bucket is empty → request is queued, not dropped

Configurable burst allowance (e.g., 2x RPS for 5 seconds)

4.1.3 Virtual Waiting Room

Triggered when the queue depth exceeds the threshold

User receives a branded waiting room page (customizable HTML/CSS), real-time position update via WebSocket or SSE

Fair FIFO queue — no position jumping

SEO/Bot bypass: verified Google/Bing crawlers skip the queue, X-Forwarded-For header preserved for origin server IP logging

4.1.4 Circuit Breaker

Protects origin server from overload cascades:

States: CLOSED (normal) → OPEN (blocking) → HALF-OPEN (testing) Opens when origin error rate exceeds 50% in a 10s window

Auto-recovery: sends 10% traffic to test origin health

Customer notified via webhook/email when circuit opens

4.2 Multi-Tenant Architecture

Every Redis key prefixed with tenant:{tenantId}:

PostgreSQL Row-Level Security (RLS) enforced per tenant

Kubernetes NetworkPolicy isolates tenant namespaces

Audit log for all config changes per tenant

4.3 Dashboard (Single-Page Nerve Center)

4.3.1 Live Metrics

Metric	Display	Data Source	Update Rate
Live visitors	Animated counter	Redis pub/sub	1s
Queue depth	Live counter + chart	Redis queue length	500ms
System health	Pulsing status icon	Circuit breaker state	2s
Protected revenue	Currency counter	PostgreSQL + Stripe	5s
RPS (current)	Gauge/speedometer	Prometheus metrics	1s
Origin response time	Line chart	Go proxy latency	1s

4.3.2 Dashboard Features

Site Switcher: dropdown to switch between multiple protected domains

Inline RPS config: update rate limits without page reload

Waiting Room Editor: live preview of custom queue page

Analytics: 7/30/90 day traffic charts, peak hour analysis

Event Mode: activate/deactivate dedicated event protection per site

Alert settings: webhook URL, email, Slack integration

4.4 Billing & Subscription

Stripe Billing for subscription management

Usage metering: request count reported to Stripe every hour via BullMQ

Overage alerts: email + dashboard warning at 80% and 95% plan limit

Plan upgrade/downgrade self-service via dashboard

Invoice history, downloadable PDF

Event add-on purchase flow: one-click activation

5. Project Folder Structure

Monorepo using pnpm workspaces + Turborepo. Three apps (proxy, api, dashboard) plus shared packages.

```
EDGEVIA/
├── apps/
│   ├── proxy/                  # Go — Core Proxy Engine[cite: 1]
│   │   ├── cmd/
│   │   │   └── server/
│   │   │       └── main.go        # Entry point[cite: 1]
│   │   ├── internal/
│   │   │   ├── proxy/            # Reverse proxy handler[cite: 1]
│   │   │   │   ├── handler.go
│   │   │   │   ├── middleware.go
│   │   │   │   └── transport.go
│   │   │   ├── ratelimit/        # Token Bucket algorithm[cite: 1]
│   │   │   │   ├── bucket.go
│   │   │   │   └── redis.go
│   │   │   ├── queue/            # Virtual Waiting Room[cite: 1]
│   │   │   │   ├── room.go
│   │   │   │   ├── position.go
│   │   │   │   └── ssehandler.go
│   │   │   ├── circuitbreaker/    # Circuit breaker pattern[cite: 1]
│   │   │   │   └── breaker.go
│   │   │   ├── ssl/              # Let's Encrypt automation[cite: 1]
│   │   │   │   ├── manager.go
│   │   │   │   └── cache.go
│   │   │   ├── tenant/           # Tenant isolation[cite: 1]
│   │   │   │   ├── resolver.go
│   │   │   │   └── config.go
│   │   │   ├── metrics/          # Prometheus metrics[cite: 1]
│   │   │   │   └── prometheus.go
```

```
│   │   │   └── logger/          # Zap structured logging[cite: 1]
│   │   │       └── logger.go
│   │   ├── proto/              # gRPC proto definitions[cite: 1]
│   │   │   └── config.proto
│   │   ├── go.mod
│   │   └── Dockerfile
│   │
│   ├── api/                    # Node.js — NestJS API[cite: 1]
│   │   ├── src/
│   │   │   ├── modules/
│   │   │   │   ├── auth/         # JWT + refresh tokens[cite: 1]
│   │   │   │   │   ├── auth.controller.ts
│   │   │   │   │   ├── auth.service.ts
│   │   │   │   │   └── jwt.strategy.ts
│   │   │   │   ├── sites/        # Customer site management[cite: 1]
│   │   │   │   │   ├── sites.controller.ts
│   │   │   │   │   ├── sites.service.ts
│   │   │   │   │   └── sites.dto.ts
│   │   │   │   ├── analytics/     # Traffic analytics[cite: 1]
│   │   │   │   │   ├── analytics.service.ts
│   │   │   │   │   └── analytics.gateway.ts
│   │   │   │   ├── billing/       # Stripe integration[cite: 1]
│   │   │   │   │   ├── billing.service.ts
│   │   │   │   │   ├── stripe.webhook.ts
│   │   │   │   │   └── metering.worker.ts
│   │   │   │   ├── tenants/       # Tenant management[cite: 1]
│   │   │   │   │   └── tenants.service.ts
│   │   │   │   ├── notifications/  # Email + webhooks[cite: 1]
│   │   │   │   │   ├── email.service.ts
│   │   │   │   │   └── webhook.service.ts
│   │   │   │   └── events/        # Event mode management[cite: 1]
│   │   │   │       └── events.service.ts
│   │   │   ├── grpc/             # gRPC client (→ Go proxy)[cite: 1]
│   │   │   │   ├── proxy.client.ts
│   │   │   │   └── config.sync.ts
│   │   │   ├── common/
│   │   │   │   ├── guards/
│   │   │   │   ├── interceptors/
│   │   │   │   └── decorators/
│   │   │   └── main.ts
│   │   ├── prisma/
│   │   │   └── schema.prisma      # Database schema[cite: 1]
│   │   ├── package.json
│   │   └── Dockerfile
│   │
│   └── dashboard/                # Next.js 14 — Frontend[cite: 1]
```

```
│       ├── app/
│       │   ├── (auth)/
│       │   │   ├── login/page.tsx
│       │   │   └── register/page.tsx
│       │   ├── (protected)/
│       │   │   ├── dashboard/page.tsx
│       │   │   ├── billing/page.tsx
│       │   │   └── settings/page.tsx
│       │   └── layout.tsx
│       ├── components/
│       │   ├── ui/              # shadcn/ui components[cite: 1]
│       │   ├── charts/          # Recharts wrappers[cite: 1]
│       │   │   ├── TrafficChart.tsx
│       │   │   ├── RpsGauge.tsx
│       │   │   └── QueueDepth.tsx
│       │   ├── dashboard/
│       │   │   ├── LivePulse.tsx
│       │   │   ├── SiteSwitcher.tsx
│       │   │   ├── StatsCard.tsx
│       │   │   └── InlineSettings.tsx
│       │   └── waiting-room/      # Waiting room editor[cite: 1]
│       │       └── WaitingRoomEditor.tsx
│       ├── hooks/
│       │   ├── useSocket.ts       # Socket.io real-time[cite: 1]
│       │   └── useMetrics.ts
│       ├── store/
│       │   └── useSiteStore.ts     # Zustand multi-site state[cite: 1]
│       ├── package.json
│       └── Dockerfile
│
├── packages/
│   ├── proto/                  # Shared .proto files[cite: 1]
│   │   └── fairflow.proto
│   └── types/                  # Shared TypeScript types[cite: 1]
│       ├── api.types.ts
│       └── metrics.types.ts
│
├── infra/                      # Infrastructure[cite: 1]
│   ├── k8s/                    # Kubernetes manifests[cite: 1]
│   │   ├── namespace.yaml
│   │   ├── proxy/
│   │   │   ├── deployment.yaml
│   │   │   ├── service.yaml
│   │   │   └── hpa.yaml          # Horizontal Pod Autoscaler[cite: 1]
│   │   ├── api/
│   │   │   ├── deployment.yaml
```

```
│   │   │   └── service.yaml
│   │   ├── dashboard/
│   │   │   └── deployment.yaml
│   │   ├── redis/
│   │   │   └── statefulset.yaml
│   │   ├── postgres/
│   │   │   └── statefulset.yaml
│   │   └── monitoring/
│   │       ├── prometheus.yaml
│   │       └── grafana.yaml
│   ├── docker-compose.yml        # Local development only[cite: 1]
│   └── terraform/                # Cloud infrastructure (future)[cite: 1]
│
├── docs/                         # Documentation[cite: 1]
│   ├── architecture.md
│   ├── api-reference.md
│   └── onboarding.md
│
├── scripts/
│   ├── setup.sh                  # One-click dev setup[cite: 1]
│   └── seed.ts                   # Database seed data[cite: 1]
│
├── pnpm-workspace.yaml           # Workspace configuration[cite: 1]
├── turbo.json                    # Turborepo configuration[cite: 1]
└── README.md
```

6. Database Schema (Key Tables)

Table	Key Fields	Purpose
tenants	id, name, plan, stripe_customer_id, created_at	One row per FairFlow customer
sites	id, tenant_id, domain, origin_url, rps_limit, active	Each protected domain
events	id, site_id, name, starts_at, ends_at, active	Scheduled event mode sessions
analytics_hourly	id, site_id, hour, req_count, queued_count, blocked_count	Aggregated traffic data

ssl_certs	id, site_id, domain, cert_pem, expires_at, renewed_at	Let's Encrypt cert cache
audit_logs	id, tenant_id, action, payload, created_at	Config change history
billing_usage	id, tenant_id, period_start, period_end, req_count, billed	Usage metering for Stripe

6.1 Redis Key Patterns

Key Pattern	Type	Purpose
tenant:{id}:bucket:{domain}	Hash	Token bucket state for rate limiting
tenant:{id}:queue:{domain}	List	Virtual waiting room queue (FIFO)
tenant:{id}:config:{domain}	Hash	Cached site config (origin URL, RPS limit)
tenant:{id}:metrics:{domain}	Stream	Real-time request metrics stream
breaker:{domain}	String	Circuit breaker state (CLOSED/OPEN/HALF_OPEN)

7. API Contracts

7.1 REST API (NestJS)

Method	Endpoint	Description	Auth
POST	/auth/register	Create new tenant account	None
POST	/auth/login	Get JWT access token	None
POST	/auth/refresh	Refresh access token	Refresh token
GET	/sites	List all sites for tenant	JWT

POST	/sites	Add new protected site	JWT
PATCH	/sites/:id/config	Update RPS limit, origin URL	JWT
DELETE	/sites/:id	Remove site from protection	JWT
GET	/analytics/:siteId	Get traffic analytics	JWT
POST	/events	Create event mode session	JWT
GET	/billing/usage	Current usage vs plan limit	JWT
POST	/billing/upgrade	Upgrade subscription plan	JWT

7.2 gRPC Service (Go ↔ Node.js)

RPC Method	Request	Response	Direction
GetSiteConfig	domain: string	SiteConfig (rps, origin, tenant_id)	Go → Node
UpdateSiteConfig	SiteConfig	Ack	Node → Go
ReportMetrics	MetricsBatch	Ack	Go → Node
GetCircuitState	domain: string	CircuitState	Go → Node

8. Implementation Roadmap

Phase	Weeks	Deliverables	Priority
Phase 1: Proxy Core	1–3	Go reverse proxy, Redis token bucket, Let's Encrypt SSL, tenant config resolver, Prometheus metrics	CRITICAL
Phase 2: API Service	4–5	NestJS API, PostgreSQL schema, Prisma, JWT auth, gRPC client, site CRUD endpoints	CRITICAL
Phase 3: Waiting Room	6–7	Virtual queue logic, SSE position updates, circuit breaker, SEO bot bypass, waiting room HTML	HIGH

		templates	
Phase 4: Dashboard	8–9	Next.js dashboard, Socket.io real-time, Zustand store, Recharts, site switcher, inline RPS config	HIGH
Phase 5: Billing	10	Stripe subscriptions, usage metering via BullMQ, event add-on purchase flow, invoice dashboard	HIGH
Phase 6: Kubernetes	11	K8s manifests, HPA for proxy, Grafana dashboards, staging + production deploy	HIGH
Phase 7: Polish	12	Onboarding flow, email notifications, waiting room editor, documentation, beta launch	MEDIUM

9. Kubernetes Architecture 9.1 Services

Service	Replicas	Resources	HPA Target
proxy (Go)	3 min → 20 max	256MB RAM, 250m CPU each	CPU > 60% → scale up
api (NestJS)	2 min → 8 max	512MB RAM, 500m CPU each	CPU > 70% → scale up
dashboard (Next.js)	2 min → 4 max	256MB RAM, 250m CPU each	CPU > 80% → scale up
redis	1 (StatefulSet)	1GB RAM (guaranteed)	Manual scale only
postgres	1 (StatefulSet)	2GB RAM, 1 CPU	Manual scale only

9.2 Traffic Flow

Internet → LoadBalancer (port 80/443) → Go Proxy Service

Go Proxy → Redis (rate limit check, queue check)

Go Proxy → Origin Server (if allowed) OR Waiting Room HTML (if queued)

Go Proxy → NestJS API via gRPC (config sync, metrics reporting)

Browser → NestJS API via Socket.io (real-time dashboard updates)

NestJS API → PostgreSQL (persistent reads/writes)

NestJS API → Redis (queue management, pub/sub)

BullMQ Workers → Stripe API (hourly usage metering)

10. Security Considerations

Threat	Mitigation
Tenant data leakage	PostgreSQL RLS + Redis key namespacing per tenant_id
JWT token theft	Short-lived access tokens (15m) + secure httpOnly refresh token cookies
DDoS at FairFlow itself	Upstream DDoS protection (Cloudflare in front of FairFlow infra)
SSL cert exposure	Certs stored encrypted in PostgreSQL, never logged
Stripe webhook spoofing	Webhook signature verification (stripe-signature header)
Bot traffic in queue	User-Agent + behavioral analysis, CAPTCHA on waiting room
gRPC interception	mTLS (mutual TLS) between proxy and API services in K8s